UNIVERSIDAD NACIONAL DEL ALTIPLANO

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

SEGUNDA UNIDAD

LENGUAJE DE PROGRAMACIÓN I

PRESENTADO POR:

DINA MILAGROS DELGADO QUISPE

CÓDIGO: 247092

PUNO — PERÚ 2026

Índice general

1. Análisis de Complejidad Computacional **5**

2. Manejo de Archivos en C/C++ **13**

3. Manejo de Múltiples Archivos **17**

4. Estructuras (`struct`) en C **23**

5. Otros Lenguajes y GUIs **27**
5.1. Comparativa de Lenguajes 27
5.2. Error clásico en Python — Tipo de dato 27
5.3. Interfaces Gráficas (GUI) 28

6. Práctica Integradora Final **29**

7. Finanálisis — Aplicación Web de Análisis Financiero Personal **33**
7.1. Descripción general del proyecto 33
7.2. Tecnologías y herramientas utilizadas 34
7.3. Arquitectura del código 36
7.4. Fundamentos de programación aplicados 36
7.5. Módulos de la aplicación 38
7.6. El cálculo de la TCEA — Método matemático 39
7.7. Diseño visual 40
7.8. Despliegue e instrucciones de uso 41
7.9. Video de presentación 41
7.10. Conclusión 42

Lista de Ejercicios **43**

Capítulo 1

Análisis de Complejidad Computacional

La **complejidad computacional** mide los recursos (tiempo y memoria) que necesita un algoritmo en función del tamaño n de su entrada, expresada con la **notación Big-O**.

Notación	Nombre	Ejemplo
$O(1)$	Constante	Acceso por índice
$O(\log n)$	Logarítmica	Búsqueda binaria
$O(n)$	Lineal	Recorrer arreglo
$O(n \log n)$	Log-lineal	Quick Sort (promedio)
$O(n^2)$	Cuadrática	Bubble Sort, bucles anidados
$O(2^n)$	Exponencial	Fibonacci recursivo

Ejercicio 1: Análisis de Complejidad Temporal — Bucles Anidados

Analice la complejidad temporal. ¿Cuántas operaciones se realizan si $n = 100$?

```cpp
#include <iostream>
#include <ctime>
using namespace std;

int main() {
    int n;
    long long suma = 0, operaciones = 0;

    cout << "Ingrese el valor de n: ";
    cin >> n;

    int** arr = new int*[n];
    for (int i = 0; i < n; i++) arr[i] = new int[n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            arr[i][j] = i + j;

    clock_t t0 = clock();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) { suma += arr[i][j]; operaciones++;
    }
    clock_t t1 = clock();

    double ms = ((double)(t1 - t0) / CLOCKS_PER_SEC) * 1000;
    cout << "Suma: " << suma << "  Operaciones: " << operaciones << endl
    ;
    cout << "Tiempo: " << ms << " ms" << endl;
    cout << "Complejidad: O(n^2)  => " << n << "x" << n << " = "
         << operaciones << " ops" << endl;

```

```
    for (int i = 0; i < n; i++) delete[] arr[i];
    delete[] arr;
    return 0;
}
```

Análisis: Complejidad $O(n^2)$. Para $n = 100$: $100 \times 100 = \mathbf{10{,}000}$ operaciones.

Ejercicio 2: Búsqueda Binaria — $O(\log n)$

Si tiene un arreglo ordenado de 1,000,000 elementos, ¿cuántas comparaciones máximas necesita?

```
#include <iostream>
#include <ctime>
using namespace std;

int busqueda_binaria(int arr[], int n, int x, int& comp) {
    int izq = 0, der = n - 1;
    while (izq <= der) {
        int medio = izq + (der - izq) / 2;
        comp++;
        if (arr[medio] == x) return medio;
        if (arr[medio] < x)  izq = medio + 1;
        else                 der = medio - 1;
    }
    return -1;
}

int main() {
    int n = 1000000;
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = i + 1;

    int x, comp = 0;
    cout << "Arreglo de 1,000,000 elementos. Buscar: ";
    cin >> x;

    clock_t t0 = clock();
    int res = busqueda_binaria(arr, n, x, comp);
    clock_t t1 = clock();

    double ms = ((double)(t1 - t0) / CLOCKS_PER_SEC) * 1000;
    if (res != -1) cout << "Encontrado en posicion: " << res << endl;
    else           cout << "No encontrado" << endl;
    cout << "Comparaciones: " << comp << "  Tiempo: " << ms << " ms" << endl;
    cout << "Complejidad: O(log n)  => maximo ~20 comparaciones" << endl;

    delete[] arr;
    return 0;
}
```

Análisis: $O(\log n)$. Para $n = 1{,}000{,}000$: $\log_2(1{,}000{,}000) \approx \mathbf{20}$ comparaciones máximas.

Ejercicio 3: Comparación de Tres Algoritmos

¿Cuál es más eficiente para $n = 10{,}000$?

```
#include <iostream>
#include <ctime>
using namespace std;
```

```cpp

int main() {
    int n = 10000;
    long long sumA = 0, sumB = 0, sumC = 0;

    // Codigo A: O(n)
    clock_t t0 = clock();
    for (int i = 0; i < n; i++) sumA += i;
    double tA = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    // Codigo B: O(n^2)
    t0 = clock();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++) sumB += j;
    double tB = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    // Codigo C: O(1)
    t0 = clock();
    sumC = (long long)n * (n + 1) / 2;
    double tC = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "Codigo A: " << tA << " ms  O(n)   = " << n << " ops" << endl;
    cout << "Codigo B: " << tB << " ms  O(n^2) = "
         << (long long)n*(n-1)/2 << " ops" << endl;
    cout << "Codigo C: " << tC << " ms  O(1)   = 1 op" << endl;
    cout << "MAS EFICIENTE: Codigo C" << endl;
    return 0;
}
```

Análisis: C es $O(1)$ (fórmula directa), A es $O(n)$, B es $O(n^2)$.

Ejercicio 4: Fibonacci Recursivo — $O(2^n)$

```cpp
#include <iostream>
#include <ctime>
using namespace std;

int fibonacci(int n, long long& llamadas) {
    llamadas++;
    if (n <= 1) return n;
    return fibonacci(n-1, llamadas) + fibonacci(n-2, llamadas);
}

int main() {
    int n;
    cout << "Ingrese n: ";
    cin >> n;

    long long llamadas = 0;
    clock_t t0 = clock();
    int res = fibonacci(n, llamadas);
    double ms = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "Fibonacci(" << n << ") = " << res << endl;
    cout << "Llamadas recursivas: " << llamadas << endl;
    cout << "Tiempo: " << ms << " ms" << endl;
    cout << "Complejidad: O(2^n) - Exponencial" << endl;
    return 0;
}
```

Análisis: $O(2^n)$. Recalcula los mismos valores repetidamente. Para $n = 30$: > 2 millones

de llamadas.

Ejercicio 5: Eliminar Duplicados — Complejidad Espacial y Temporal

```
#include <iostream>
#include <ctime>
using namespace std;

int eliminar_duplicados(int arr[], int n) {
    int temp[n];
    int k = 0;
    for (int i = 0; i < n; i++) {
        int dup = 0;
        for (int j = 0; j < k; j++)
            if (arr[i] == temp[j]) { dup = 1; break; }
        if (!dup) temp[k++] = arr[i];
    }
    return k;
}

int main() {
    int n;
    cout << "Tamano del arreglo: ";
    cin >> n;
    int* arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = rand() % (n/2);

    clock_t t0 = clock();
    int k = eliminar_duplicados(arr, n);
    double ms = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "Elementos originales: " << n << "  Unicos: " << k << endl;
    cout << "Tiempo: " << ms << " ms" << endl;
    cout << "Complejidad Temporal: O(n^2)  Espacial: O(n)" << endl;

    delete[] arr;
    return 0;
}
```

Análisis: Temporal $O(n^2)$ por los dos bucles anidados; espacial $O(n)$ por el arreglo auxiliar.

Ejercicio 6: Bucle Logarítmico

```
#include <iostream>
#include <ctime>
using namespace std;

int main() {
    int n;
    cout << "Ingrese n: ";
    cin >> n;

    int count = 0;
    clock_t t0 = clock();
    for (int i = 1; i < n; i = i * 2) count++;
    double ms = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "n = " << n << "  Iteraciones: " << count << endl;
    cout << "Tiempo: " << ms << " ms" << endl;
    cout << "Complejidad: O(log n)     secuencia: 1,2,4,8,16..." << endl
    ;
    return 0;
```

```
}
```

Análisis: $O(\log n)$. El índice se duplica en cada paso. Para $n = 1024$: 10 iteraciones.

Ejercicio 7: Bubble Sort vs Quick Sort

```
#include <iostream>
#include <ctime>
using namespace std;

void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (arr[j] > arr[j+1]) { int t=arr[j]; arr[j]=arr[j+1]; arr[j+1]=t; }
}

void quicksort(int arr[], int izq, int der) {
    if (izq < der) {
        int piv = arr[der], i = izq - 1;
        for (int j = izq; j < der; j++)
            if (arr[j] < piv) { i++; int t=arr[i]; arr[i]=arr[j]; arr[j]=t; }
        int t=arr[i+1]; arr[i+1]=arr[der]; arr[der]=t;
        int pi = i + 1;
        quicksort(arr, izq, pi-1);
        quicksort(arr, pi+1, der);
    }
}

int main() {
    int n = 10000;
    int* a1 = new int[n]; int* a2 = new int[n];
    for (int i = 0; i < n; i++) { a1[i] = rand()%10000; a2[i] = a1[i]; }

    clock_t t0 = clock();
    bubble_sort(a1, n);
    double tB = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    t0 = clock();
    quicksort(a2, 0, n-1);
    double tQ = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "Bubble Sort: " << tB << " ms  O(n^2)" << endl;
    cout << "Quick Sort:  " << tQ << " ms  O(n log n) promedio" << endl;

    delete[] a1; delete[] a2;
    return 0;
}
```

Algoritmo	Peor caso	Promedio
Bubble Sort	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n^2)$ (arreglo ordenado)	$O(n \log n)$

Ejercicio 8: Verificar Número Primo — $O(\sqrt{n})$

```
#include <iostream>
#include <ctime>
```

```
using namespace std;

int es_primo(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}

int main() {
    int n;
    cout << "Ingrese un numero: ";
    cin >> n;

    clock_t t0 = clock();
    int res = es_primo(n);
    double ms = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << n << (res ? " ES primo" : " NO es primo") << endl;
    cout << "Tiempo: " << ms << " ms  Complejidad: O(sqrt(n))" << endl;
    return 0;
}
```

Análisis: El bucle itera hasta $\sqrt{n}$ porque si n tiene divisor $> \sqrt{n}$, el complementario ya fue revisado.

Ejercicio 9: Multiplicación de Matrices — $O(n^3)$

```
#include <iostream>
#include <ctime>
using namespace std;

void multiplicar(int** A, int** B, int** C, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            C[i][j] = 0;
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
}

int main() {
    int n;
    cout << "Tamano n: ";
    cin >> n;

    int** A = new int*[n]; int** B = new int*[n]; int** C = new int*[n];
    for (int i=0;i<n;i++){A[i]=new int[n];B[i]=new int[n];C[i]=new int[n
    ];}
    for (int i=0;i<n;i++)
        for (int j=0;j<n;j++){A[i][j]=rand()%10;B[i][j]=rand()%10;}

    clock_t t0 = clock();
    multiplicar(A, B, C, n);
    double ms = ((double)(clock()-t0)/CLOCKS_PER_SEC)*1000;

    cout << "Tiempo: " << ms << " ms  Complejidad: O(n^3)" << endl;
    cout << "Operaciones: " << (long long)n*n*n << endl;
    cout << "Strassen reduce a O(n^2.807)" << endl;

    for(int i=0;i<n;i++){delete[]A[i];delete[]B[i];delete[]C[i];}
    delete[]A;delete[]B;delete[]C;
```

```
    return 0;
}
```

Análisis: Tres bucles anidados $\Rightarrow O(n^3)$. Strassen $\approx O(n^{2,807})$ mejora el rendimiento para matrices grandes.

Ejercicio 10: Tabla de Complejidades por Estructura de Datos

```
#include <iostream>
using namespace std;
int main() {
    cout << "Operacion            | Array  | Lista  | Hash Table" << endl;
    cout << "-----------------------------------------------------" << endl;
    cout << "Acceso por indice    | O(1)   | O(n)   | N/A       " << endl;
    cout << "Busqueda             | O(n)   | O(n)   | O(1) prom " << endl;
    cout << "Insercion al inicio  | O(n)   | O(1)   | O(1) prom " << endl;
    cout << "Insercion al final   | O(1)   | O(1)*  | O(1) prom " << endl;
    cout << "Eliminacion          | O(n)   | O(n)   | O(1) prom " << endl;
    cout << "* Con puntero al ultimo nodo" << endl;
    return 0;
}
```

Operación	Array	Lista Enlazada	Hash Table
Acceso por índice	$O(1)$	$O(n)$	N/A
Búsqueda	$O(n)$	$O(n)$	$O(1)$ prom.
Inserción al inicio	$O(n)$	$O(1)$	$O(1)$ prom.
Inserción al final	$O(1)$	$O(1)$*	$O(1)$ prom.
Eliminación	$O(n)$	$O(n)$	$O(1)$ prom.

*Con puntero al último nodo.

Capítulo 2

Manejo de Archivos en C/C++

Los modos de apertura más usados son:

Modo	Descripción
`"w"` / `ios::out`	Escritura — crea o sobrescribe
`r"` / `ios::in`	Lectura — el archivo debe existir
`.ª"` / `ios::app`	Append — agrega al final
`"wb"` / `ios::binary`	Escritura binaria
`rb"`	Lectura binaria

Ejercicio 11: Sistema de Registro de Temperaturas

Solicitar 5 temperaturas, guardarlas en archivo, calcular estadísticas y agregarlas con *append*.

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    float temperaturas[5];
    float suma = 0, promedio, maxima, minima;

    cout << "=== REGISTRO DE TEMPERATURAS ===" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "Temperatura " << (i+1) << ": ";
        cin >> temperaturas[i];
    }

    // Escribir mediciones
    ofstream fw("temperaturas.txt");
    fw << "MEDICIONES:" << endl;
    for (int i = 0; i < 5; i++)
        fw << fixed << setprecision(1) << temperaturas[i] << endl;
    fw.close();

    // Leer y calcular estadisticas
    ifstream fr("temperaturas.txt");
    string linea;
    getline(fr, linea);
    float temp; int count = 0;
    maxima = -999; minima = 999;
    while (fr >> temp) {
        suma += temp; count++;
        if (temp > maxima) maxima = temp;
        if (temp < minima) minima = temp;
    }
```

```
    fr.close();
    promedio = suma / count;

    // Agregar estadisticas
    ofstream fa("temperaturas.txt", ios::app);
    fa << "\nESTADISTICAS:" << endl;
    fa << fixed << setprecision(2);
    fa << "Promedio: " << promedio << endl;
    fa << "Maxima: " << maxima << endl;
    fa << "Minima: " << minima << endl;
    fa.close();

    cout << fixed << setprecision(2);
    cout << "Promedio: " << promedio << " C" << endl;
    cout << "Maxima: "   << maxima   << " C" << endl;
    cout << "Minima: "   << minima   << " C" << endl;
    return 0;
}
```

Salida esperada

```
MEDICIONES:
23.5   25.1   24.8   26.2   23.9
ESTADISTICAS:
Promedio: 24.70   Maxima: 26.20   Minima: 23.50
```

Ejercicio 12: Base de Datos de Productos (archivo binario)

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cstring>
using namespace std;

struct Producto { int codigo; char nombre[30]; float precio; int stock;
    };

void agregar() {
    Producto p;
    cout << "Codigo: "; cin >> p.codigo; cin.ignore();
    cout << "Nombre: "; cin.getline(p.nombre, 30);
    cout << "Precio: "; cin >> p.precio;
    cout << "Stock: ";  cin >> p.stock;
    ofstream f("productos.dat", ios::binary | ios::app);
    f.write(reinterpret_cast<char*>(&p), sizeof(Producto));
    f.close();
    cout << "Producto agregado!" << endl;
}

void listar() {
    Producto p;
    ifstream f("productos.dat", ios::binary);
    if (!f) { cout << "Sin productos." << endl; return; }
    cout << left << setw(8) << "Cod" << setw(22) << "Nombre"
         << setw(10) << "Precio" << "Stock" << endl;
    cout << string(50, '-') << endl;
    while (f.read(reinterpret_cast<char*>(&p), sizeof(Producto)))
        cout << left << setw(8) << p.codigo << setw(22) << p.nombre
             << setw(10) << fixed << setprecision(2) << p.precio
             << p.stock << endl;
    f.close();
}
```

```

void buscar() {
    int cod; Producto p; bool ok = false;
    cout << "Codigo a buscar: "; cin >> cod;
    ifstream f("productos.dat", ios::binary);
    while (f.read(reinterpret_cast<char*>(&p), sizeof(Producto)))
        if (p.codigo == cod) {
            ok = true;
            cout << "Nombre: " << p.nombre << "  Precio: S/" << p.precio
                << "  Stock: " << p.stock << endl;
            break;
        }
    f.close();
    if (!ok) cout << "No encontrado." << endl;
}

void actualizar() {
    int cod, nueva; Producto p; bool ok = false;
    cout << "Codigo: "; cin >> cod;
    fstream f("productos.dat", ios::binary | ios::in | ios::out);
    while (f.read(reinterpret_cast<char*>(&p), sizeof(Producto)))
        if (p.codigo == cod) {
            ok = true;
            cout << "Stock actual: " << p.stock << "  Nuevo: "; cin >>
    nueva;
            p.stock = nueva;
            f.seekp(-(int)sizeof(Producto), ios::cur);
            f.write(reinterpret_cast<char*>(&p), sizeof(Producto));
            cout << "Actualizado!" << endl;
            break;
        }
    f.close();
    if (!ok) cout << "No encontrado." << endl;
}

int main() {
    int op;
    do {
        cout << "\n1.Agregar  2.Listar  3.Buscar  4.Actualizar  5.Salir\
    nOpcion: ";
        cin >> op;
        if (op==1) agregar();
        else if (op==2) listar();
        else if (op==3) buscar();
        else if (op==4) actualizar();
    } while (op != 5);
    return 0;
}
```

Ejercicio 13: Lectura de 100 Archivos — Promedio de Edades

```
#include <stdio.h>

int main() {
    FILE *f;
    int edad, suma = 0;
    char nombre[50];

    for (int i = 1; i <= 100; i++) {
        sprintf(nombre, "alumnos_data/alumno_%d.txt", i);
        f = fopen(nombre, "r");
        fscanf(f, "%d", &edad);
        printf("archivo %s  edad: %d\n", nombre, edad);
```

```
        suma += edad;
        fclose(f);
    }

    printf("Promedio: %.2f\n", (float)suma / 100);
    return 0;
}
```

Ejercicio 14: Lectura de Archivos por Bloques de 10

```
#include <stdio.h>

int main() {
    int edad, suma;

    for (int bloque = 1; bloque <= 10; bloque++) {
        suma = 0;
        printf("BLOQUE %d:\n", bloque);

        for (int j = (bloque-1)*10 + 1; j <= bloque*10; j++) {
            char nombre[30];
            sprintf(nombre, "alumnos_data/alumno_%d.txt", j);
            FILE *f = fopen(nombre, "r");
            if (f && fscanf(f, "%d", &edad) == 1) {
                printf("  Alumno %d: %d\n", j, edad);
                suma += edad;
            }
            if (f) fclose(f);
        }
        printf("Promedio del bloque: %.2f\n\n", suma / 10.0);
    }
    printf("Elaborado por: Dina\n");
    return 0;
}
```

Capítulo 3

Manejo de Múltiples Archivos

Patrón básico:

```
for (int i = 1; i <= N; i++) {
    sprintf(nombre, "archivo_%d.txt", i);  // nombre dinamico
    archivo = fopen(nombre, "modo");        // abrir
    // leer o escribir
    fclose(archivo);                        // cerrar
}
```

Ejercicio 15: Registro de Ventas Diarias

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ofstream archivo;
    char nombre[30];
    float ventas[] = {1500.50, 2300.75, 1800.00};

    for (int dia = 1; dia <= 3; dia++) {
        sprintf(nombre, "venta_dia_%d.txt", dia);
        archivo.open(nombre);
        archivo << fixed << setprecision(2) << ventas[dia-1] << endl;
        archivo.close();
    }

    cout << "3 archivos de ventas creados" << endl;
    return 0;
}
```

Ejercicio 16: Total de Ventas desde Múltiples Archivos

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ifstream archivo;
    char nombre[30];
    float venta, total = 0;

    for (int dia = 1; dia <= 3; dia++) {
        sprintf(nombre, "venta_dia_%d.txt", dia);
        archivo.open(nombre);
        archivo >> venta;
        total += venta;
        archivo.close();
    }
```

```

    cout << fixed << setprecision(2);
    cout << "Total de ventas: " << total << endl;
    return 0;
}
```

Salida esperada

```
Total de ventas: 5600.25
```

Ejercicio 17: Temperaturas por Hora en Archivos Separados

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ofstream archivo;
    char nombre[30];
    float temps[] = {18.5, 19.2, 20.1, 21.5};

    for (int h = 0; h < 4; h++) {
        sprintf(nombre, "temp_hora_%d.txt", h);
        archivo.open(nombre);
        archivo << "Hora " << h << ": " << fixed << setprecision(1)
                << temps[h] << "C" << endl;
        archivo.close();
    }

    cout << "Temperaturas guardadas" << endl;
    return 0;
}
```

Ejercicio 18: Temperatura Máxima del Día

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ifstream archivo;
    char nombre[30], buffer[20];
    int hora_actual, hora_max = 0;
    float temp, temp_max = -999;

    for (int h = 0; h < 4; h++) {
        sprintf(nombre, "temp_hora_%d.txt", h);
        archivo.open(nombre);
        archivo >> buffer >> hora_actual >> buffer >> temp;
        if (temp > temp_max) { temp_max = temp; hora_max = hora_actual;
    }
        archivo.close();
    }

    cout << fixed << setprecision(1);
    cout << "Hora mas calurosa: " << hora_max
         << " (" << temp_max << "C)" << endl;
    return 0;
}
```

Ejercicio 19: Logs de Errores por Tipo

```
#include <iostream>
#include <fstream>
using namespace std;

void registrar_error(const char* tipo, const char* mensaje) {
    ofstream archivo;
    char nombre[40];
    sprintf(nombre, "log_%s.txt", tipo);
    archivo.open(nombre, ios::app);
    archivo << mensaje << endl;
    archivo.close();
}

int main() {
    registrar_error("red",   "Conexion perdida");
    registrar_error("disco", "Espacio insuficiente");
    registrar_error("red",   "Timeout en servidor");
    cout << "Errores registrados" << endl;
    return 0;
}
```

Ejercicio 20: Contar Errores en Log

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream archivo("log_red.txt");
    string linea;
    int contador = 0;

    while (getline(archivo, linea)) contador++;
    archivo.close();

    cout << "Errores de red: " << contador << endl;
    return 0;
}
```

Ejercicio 21: Backup de Archivo

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream origen("datos.txt");
    ofstream destino("datos_backup.txt");
    char c;

    while (origen.get(c)) destino.put(c);

    origen.close(); destino.close();
    cout << "Backup completado" << endl;
    return 0;
}
```

Ejercicio 22: Filtrar Temperaturas Altas

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ifstream entrada("todas_temps.txt");
    ofstream salida("temps_altas.txt");
    float temp;

    while (entrada >> temp)
        if (temp > 20.0)
            salida << fixed << setprecision(1) << temp << endl;

    entrada.close(); salida.close();
    cout << "Filtrado completado" << endl;
    return 0;
}
```

Ejercicio 23: Consolidar Reportes de Sucursales

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream entrada;
    ofstream salida("reporte_total.txt");
    char nombre[30];
    string linea;

    for (int s = 1; s <= 3; s++) {
        sprintf(nombre, "sucursal_%d.txt", s);
        entrada.open(nombre);
        while (getline(entrada, linea)) salida << linea << endl;
        entrada.close();
    }

    salida.close();
    cout << "Reportes consolidados" << endl;
    return 0;
}
```

Ejercicio 24: Procesar CSV de Calificaciones

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {
    ifstream archivo("notas.csv");
    char nombre[50];
    float n1, n2, n3;
    string linea;

    getline(archivo, linea); // saltar encabezado

    while (archivo.getline(nombre, 50, ',')) {
        char c;
        archivo >> n1 >> c >> n2 >> c >> n3;
```

```
17          archivo.ignore();
18          cout << nombre << ": " << fixed << setprecision(2)
19               << (n1 + n2 + n3) / 3 << endl;
20      }
21
22      archivo.close();
23      return 0;
24  }
```

Capítulo 4

Estructuras (struct) en C

Una **estructura** agrupa variables de diferentes tipos bajo un mismo nombre.

```c
typedef struct {
    tipo campo1;
    tipo campo2;
} NombreStruct;

NombreStruct var;        // declarar
var.campo1 = valor;      // acceder con punto (.)
strcpy(var.nombre, "texto");  // copiar cadenas
```

Ejercicio 25: Ficha de Mascota

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char  nombre[30];
    char  especie[20];
    int   edad;
    float peso;
} Mascota;

int main() {
    Mascota m1;
    strcpy(m1.nombre,  "Firulais");
    strcpy(m1.especie, "Perro");
    m1.edad = 3;
    m1.peso = 8.5;

    printf("Nombre:  %s\n",    m1.nombre);
    printf("Especie: %s\n",    m1.especie);
    printf("Edad:    %d anos\n", m1.edad);
    printf("Peso:    %.2f kg\n", m1.peso);
    return 0;
}
```

Salida esperada

```
Nombre: Firulais   Especie: Perro   Edad: 3 anos   Peso: 8.50 kg
```

Ejercicio 26: Registro de Temperatura

```c
#include <stdio.h>

typedef struct {
    int   hora;
    float temperatura;
    char  ciudad[20];
} Registro;

```

```c
int main() {
    Registro r[3] = {{8,15.2,"Puno"},{12,18.5,"Puno"},{18,16.8,"Puno"}};
    float suma = 0;

    for (int i = 0; i < 3; i++) {
        printf("Hora %02d: %.1fC - %s\n",
               r[i].hora, r[i].temperatura, r[i].ciudad);
        suma += r[i].temperatura;
    }
    printf("Promedio: %.2fC\n", suma / 3);
    return 0;
}
```

Salida esperada

```
Hora 08: 15.2C - Puno   Hora 12: 18.5C - Puno   Hora 18: 16.8C - Puno
Promedio: 16.83C
```

Ejercicio 27: Salón de Clases — Promedio y Mejor Alumno

```c
#include <stdio.h>

typedef struct { char nombre[30]; float nota; } Alumno;

int main() {
    Alumno s[5] = {{"Carlos",14.5},{"Maria",18.0},
                   {"Pedro",12.0},{"Lucia",19.5},{"Juan",16.0}};
    float suma = 0; int mejor = 0;

    for (int i = 0; i < 5; i++) {
        printf("%-10s: %.1f\n", s[i].nombre, s[i].nota);
        suma += s[i].nota;
        if (s[i].nota > s[mejor].nota) mejor = i;
    }
    printf("Promedio del salon: %.2f\n", suma / 5);
    printf("Mejor alumno: %s con %.1f\n", s[mejor].nombre, s[mejor].nota
    );
    return 0;
}
```

Salida esperada

```
Promedio del salon: 16.00   Mejor alumno: Lucia con 19.5
```

Ejercicio 28: Inventario con Stock Bajo

```c
#include <stdio.h>

typedef struct { char nombre[30]; float precio; int stock; } Producto;

int main() {
    Producto inv[4] = {{"Cuaderno",5.50,50},{"Lapicero",1.20,8},
                       {"Borrador",0.80,5}, {"Regla 30cm",3.00,20}};
    float total = 0;

    for (int i = 0; i < 4; i++) {
        printf("%-12s S/%.2f  Stock: %d\n",
               inv[i].nombre, inv[i].precio, inv[i].stock);
        total += inv[i].precio * inv[i].stock;
    }
    printf("Valor total: S/%.2f\n", total);
```

```
    printf("Productos con stock bajo:\n");
    for (int i = 0; i < 4; i++)
        if (inv[i].stock < 10)
            printf("  %-12s Stock: %d\n", inv[i].nombre, inv[i].stock);
    return 0;
}
```

Salida esperada

```
Valor total: S/358.60   Lapicero Stock: 8   Borrador Stock: 5
```

Ejercicio 29: Boleta de Pago con Funciones

```
#include <stdio.h>

typedef struct { char nombre[30]; int codigo; float sueldo_bruto; }
    Empleado;

float calcular_neto(Empleado e) {
    return e.sueldo_bruto * (1 - 0.13f);
}

void imprimir_boleta(Empleado e) {
    float desc = e.sueldo_bruto * 0.13f;
    float neto = calcular_neto(e);
    printf("============================\n");
    printf("        BOLETA DE PAGO\n");
    printf("============================\n");
    printf("Nombre: %s\n", e.nombre);
    printf("Codigo: %d\n", e.codigo);
    printf("Sueldo Bruto:  S/%.2f\n", e.sueldo_bruto);
    printf("Descuento AFP: S/%.2f\n", desc);
    printf("Sueldo Neto:   S/%.2f\n", neto);
    printf("============================\n");
}

int main() {
    Empleado emp = {"Rosa Quispe", 1042, 2800.0};
    imprimir_boleta(emp);
    return 0;
}
```

Salida esperada

```
Sueldo Bruto: S/2800.00   Descuento AFP: S/364.00   Sueldo Neto: S/2436.00
```

Ejercicio 30: Sensores en Archivo de Texto

```
#include <stdio.h>

typedef struct { int id; char ubicacion[20]; float temperatura; float
    humedad; } Sensor;

int main() {
    Sensor s[3] = {{1,"Sala A",22.5,60.0},{2,"Sala B",25.1,55.0},
                   {3,"Exterior",18.3,75.0}};

    FILE *f = fopen("sensores.txt", "w");
    for (int i = 0; i < 3; i++)
        fprintf(f, "%d,%s,%.1f,%.1f\n",
```

```
                s[i].id, s[i].ubicacion, s[i].temperatura, s[i].humedad);
    fclose(f);

    Sensor r; float suma = 0; int n = 0;
    f = fopen("sensores.txt", "r");
    printf("%-4s %-12s %-6s %-8s\n","ID","Ubicacion","Temp","Humedad");
    printf("--------------------------------\n");
    while (fscanf(f, "%d,%[^,],%f,%f\n",
                &r.id, r.ubicacion, &r.temperatura, &r.humedad) == 4)
    {
        printf("%-4d %-12s %-6.1f %.1f%%\n",
               r.id, r.ubicacion, r.temperatura, r.humedad);
        suma += r.temperatura; n++;
    }
    fclose(f);
    printf("--------------------------------\n");
    printf("Temperatura promedio: %.2fC\n", suma / n);
    return 0;
}
```

Salida esperada

```
1 Sala A 22.5 60.0%   2 Sala B 25.1 55.0%   3 Exterior 18.3 75.0%
Temperatura promedio: 21.97C
```

Capítulo 5

Otros Lenguajes y GUIs

5.1 Comparativa de Lenguajes

Lenguaje	Fortaleza	Uso típico
PHP	Backend web	WordPress, Laravel
Java	Multiplataforma	Android, empresarial
JavaScript	Frontend y backend	Web, Node.js
Go	Concurrencia	Microservicios, cloud
Rust	Seguridad memoria	Sistemas, WebAssembly
Mojo	IA / velocidad	Machine Learning

5.2 Error clásico en Python — Tipo de dato

```python
# MAL: suma concatena strings
num1 = input("Primer numero: ")
num2 = input("Segundo numero: ")
resultado = num1 + num2        # "23" + "5" = "235"
print("la suma es:", resultado)

# BIEN: convertir a float
num1 = float(input("Primer numero: "))
num2 = float(input("Segundo numero: "))
print("la suma es:", num1 + num2)
```

5.3 Interfaces Gráficas (GUI)

Lenguaje	Framework	Características
Python	Tkinter	Incluido con Python, simple
Python	PyQt	Potente, basado en Qt
Python	Kivy	Multiplataforma, mobile
Python	wxPython	Apariencia nativa del SO
Java	Swing	Clásico, parte del JDK
Java	JavaFX	Moderno, FXML, CSS
C++	Qt	Muy potente, multiplataforma
C / C++	GTK	Linux / GNOME
C#	Windows Forms	Rápido, solo Windows
C#	WPF	XAML, enlace de datos
JavaScript	Electron	Apps escritorio con web
Dart	Flutter	Mobile, web y escritorio
Rust	Iced	Moderno, reactivo

Instalación de Streamlit (Python)

```
python -m venv venv          # crear entorno virtual
venv\Scripts\activate        # activar (Windows)
pip install streamlit        # instalar
streamlit --version          # verificar
streamlit run suma.py        # ejecutar
```

```python
import streamlit as st

st.title("Calculadora de Suma")
num1 = st.number_input("Primer numero:", value=0.0)
num2 = st.number_input("Segundo numero:", value=0.0)

if st.button("Calcular"):
    st.success(f"La suma es: {num1 + num2}")
```

Listing 5.1: App de suma con Streamlit

Capítulo 6

Práctica Integradora Final

Ejercicio 31: Sistema de Gestión de Biblioteca (completar código)

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    int  codigo; char titulo[50]; char autor[30]; int disponible;
} Libro;

int main() {
    Libro lib[5] = {
        {101,"Cien Anos de Soledad","Garcia Marquez",1},
        {102,"Don Quijote","Cervantes",1},
        {103,"El Principito","Saint-Exupery",1},
        {104,"1984","Orwell",1},
        {105,"Rayuela","Cortazar",1}
    };

    FILE *f; char nombre[30]; int cod;

    for (int i = 1; i <= 3; i++) {
        sprintf(nombre, "prestamo_%d.txt", i);    // <-- completar
        f = fopen(nombre, "r");                   // <-- completar
        if (f != NULL) {
            fscanf(f, "%d", &cod);                // <-- completar
            for (int j = 0; j < 5; j++)
                if (lib[j].codigo == cod)         // <-- completar
                    lib[j].disponible = 0;
            fclose(f);
        }
    }

    printf("LIBROS DISPONIBLES:\n");
    printf("%-5s %-30s %-20s\n", "Cod", "Titulo", "Autor");
    printf("-------------------------------------------------------\n");

    int disp = 0;
    for (int i = 0; i < 5; i++)
        if (lib[i].disponible == 1) {             // <-- completar
            printf("%-5d %-30s %-20s\n",
                   lib[i].codigo, lib[i].titulo, lib[i].autor);
            disp++;
        }

    float porc = (3.0f / 5) * 100;               // <-- completar
    printf("\nLibros prestados: %.1f%%\n", porc);
    return 0;
}
```

Archivos de entrada: prestamo_1.txt=102, prestamo_2.txt=104, prestamo_3.txt=101

```
103 El Principito Saint-Exupery
105 Rayuela Cortazar
Libros prestados: 60.0%
```

Ejercicio 32: Análisis de Temperaturas Mensuales (completar código)

```c
#include <stdio.h>

float calcular_promedio(float t[], int n) {
    float s = 0;
    for (int i = 0; i < n; i++) s += t[i];  // <-- completar
    return s / n;                            // <-- completar
}

float encontrar_maximo(float t[], int n) {
    float m = t[0];
    for (int i = 1; i < n; i++)
        if (t[i] > m) m = t[i];             // <-- completar
    return m;
}

float encontrar_minimo(float t[], int n) {   // <-- implementar
    float m = t[0];
    for (int i = 1; i < n; i++)
        if (t[i] < m) m = t[i];
    return m;
}

int contar_calurosos(float t[], int n) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (t[i] > 25.0) c++;               // <-- completar
    return c;
}

int main() {
    float temps[30]; int total = 0;
    FILE *ent = fopen("temperaturas.txt", "r");
    while (fscanf(ent, "%f", &temps[total]) == 1) total++;
    fclose(ent);

    float prom  = calcular_promedio(temps, total);   // <-- completar
    float tmax  = encontrar_maximo(temps, total);    // <-- completar
    float tmin  = encontrar_minimo(temps, total);    // <-- completar
    int   cal   = contar_calurosos(temps, total);    // <-- completar

    FILE *sal = fopen("reporte.txt", "w");           // <-- completar
    fprintf(sal, "REPORTE DE TEMPERATURAS\n");
    fprintf(sal, "=======================\n\n");
    fprintf(sal, "Total de dias: %d\n", total);
    fprintf(sal, "Temperatura promedio: %.2fC\n", prom);
    fprintf(sal, "Temperatura maxima: %.2fC\n",   tmax);
    fprintf(sal, "Temperatura minima: %.2fC\n",   tmin);
    fprintf(sal, "Dias calurosos (>25C): %d\n",   cal);
    fclose(sal);                                     // <-- completar
    printf("Reporte generado exitosamente\n");
    return 0;
}
```

reporte.txt

```
Total de dias: 30   Promedio: 25.77C   Maxima: 29.10C   Minima: 22.50C
Calurosos: 17
```

Ejercicio 33: Corregir Sistema de Inventario — 4 Errores

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    int codigo; char nombre[30]; float precio; int stock;
} Producto;

int main() {
    Producto inv[3] = {{1,"Laptop",2500.0,10},{2,"Mouse",35.0,50},
                       {3,"Teclado",85.0,30}};
    float total = 0;

    // ERROR 1: i <= 3 accede fuera del arreglo (debe ser i < 3)
    for (int i = 0; i <= 3; i++)
        total += inv[i].precio * inv[i].stock;

    printf("Valor total: S/%.2f\n", total);

    Producto nuevo;
    nuevo.codigo = 4;
    // ERROR 2: argumentos invertidos (debe ser strcpy(nuevo.nombre, "Monitor"))
    strcpy("Monitor", nuevo.nombre);
    nuevo.precio = 450.0; nuevo.stock = 15;

    FILE *f = fopen("nuevo_producto.txt", "w");
    fprintf(f, "%d,%s,%.2f,%d\n",
            nuevo.codigo, nuevo.nombre, nuevo.precio, nuevo.stock);
    // ERROR 3: falta fclose(f)

    printf("Producto agregado al archivo\n");
    return 0;
}
```

Listing 6.1: Código con errores — encontrar y corregir

Correcciones:

1. `i <= 3` → `i <3`
2. `strcpy("Monitor", nuevo.nombre)` → `strcpy(nuevo.nombre, "Monitor")`
3. Agregar `fclose(f)` después del `fprintf`
4. (Conceptual) El arreglo tiene 3 elementos: índices válidos 0, 1, 2

Salida esperada

```
Valor total del inventario: S/30425.00
Producto agregado al archivo
```

Ejercicio 34: Refactorizar con Función `calcular_promedio()`

```c
#include <stdio.h>

```

```c
float calcular_promedio(float ventas[], int n) {
    float suma = 0;
    for (int i = 0; i < n; i++) suma += ventas[i];
    return suma / n;
}

int main() {
    float enero  [5] = {1200, 1500, 1350, 1600, 1450};
    float febrero[5] = {1400, 1650, 1500, 1700, 1550};
    float marzo  [5] = {1300, 1550, 1450, 1650, 1500};

    printf("Promedio Enero:   %.2f\n", calcular_promedio(enero,   5));
    printf("Promedio Febrero: %.2f\n", calcular_promedio(febrero, 5));
    printf("Promedio Marzo:   %.2f\n", calcular_promedio(marzo,   5));
    return 0;
}
```

Listing 6.2: Código refactorizado

Salida esperada

```
Promedio Enero: 1420.00
Promedio Febrero: 1560.00
Promedio Marzo: 1490.00
```

Capítulo 7

Finanálisis — Aplicación Web de Análisis Financiero Personal

Autora	Dina Milagros Delgado Quispe
Tecnología	Python 3 · Streamlit · Plotly · Pandas
Año	2026
URL pública	finanalisis-app.streamlit.app
YouTube	https://youtu.be/QT2V7mOsFzM
Repositorio	https://github.com/Milagros1804/finanalisis

7.1 Descripción general del proyecto

Finanálisis es una aplicación web de educación y análisis financiero personal, desarrollada en Python y desplegada en internet de forma gratuita mediante Streamlit Cloud. La aplicación está dirigida a cualquier persona que desee entender el costo real de un crédito bancario, comparar ofertas de distintas entidades financieras y llevar un registro ordenado de sus ingresos y gastos.

El proyecto nació para resolver un problema concreto: la mayoría de personas en el Perú toman decisiones financieras basadas únicamente en la **Tasa Nominal Anual (TNA)** que los bancos publicitan, sin conocer la **TCEA** (Tasa de Costo Efectivo Anual), que es el indicador que refleja el costo real total del crédito incluyendo seguros y comisiones.

> **Objetivo principal**
>
> Que cualquier persona, sin conocimientos financieros previos, pueda calcular el costo real de un crédito, comparar entidades financieras con datos verídicos y controlar su economía personal desde cualquier dispositivo con acceso a internet.

7.1.1 Resumen de características

Característica	Detalle
Módulos principales	3 (Calculadora, Comparador, Gestor de finanzas)
Lenguaje	Python 3
Interfaz	Streamlit (framework web para Python)
Gráficas	Plotly (interactivas, tema oscuro)
Exportación	CSV y Excel (.xlsx con 2 hojas)
Despliegue	Streamlit Cloud (gratuito, público)
Repositorio	GitHub
Acceso	Cualquier navegador, sin instalación
Privacidad	Datos privados por sesión de usuario
Monedas soportadas	Soles (S/), Dólares ($), Euros (EUR)

7.1.2 Diferencia entre TNA, TEA y TCEA

Los tres indicadores miden costos distintos de un crédito:

Indicador	Nombre completo	Qué incluye
TNA	Tasa Nominal Anual	Solo la tasa base. No incluye capitalización ni costos adicionales.
TEA	Tasa Efectiva Anual	TNA capitalizada mensualmente. Más precisa, pero aún sin seguros ni comisiones.
TCEA	Tasa de Costo Efectivo Anual	**El costo REAL total**: cuota base + seguros + comisiones + todos los cargos.

7.2 Tecnologías y herramientas utilizadas

7.2.1 Lenguaje de programación

Python 3 fue el lenguaje elegido por su amplio ecosistema de librerías para ciencia de datos, su sintaxis legible y su compatibilidad nativa con Streamlit. Todo el proyecto —lógica financiera, interfaz, gráficas y exportación— está en un único archivo `finanalisis.py` de aproximadamente **1,750 líneas de código**.

7.2.2 Librerías y frameworks

Librería / Herramienta	Versión mín.	Para qué se usó
Streamlit	$\geq$1.32	Framework principal. Convierte código Python en una aplicación web interactiva sin necesidad de HTML, CSS de servidor ni JavaScript. Gestiona la interfaz, la navegación entre páginas y el estado de sesión.

Librería / Herramienta	Versión mín.	Para qué se usó
Pandas	≥2.0	Manipulación de datos tabulares. Se usa para construir la tabla de amortización, filtrar movimientos financieros por fecha/categoría y preparar los datos para exportación.
Plotly	≥5.18	Gráficas interactivas con tema oscuro: gráfico donut (composición del préstamo), barras apiladas (capital vs. interés por mes), barras comparativas entre bancos y gráfica de líneas de evolución temporal de ingresos y gastos.
OpenPyXL	≥3.1	Generación de archivos Excel (.xlsx). Se usa en el módulo de exportación para crear un libro con dos hojas: movimientos detallados y resumen mensual.
NumPy	≥1.24	Soporte numérico de bajo nivel utilizado en los cálculos financieros.
Base64	stdlib	Módulo estándar de Python. Se usa para codificar la imagen `hero.png` en base64 e incrustarla directamente en el HTML de la app sin necesidad de un servidor de archivos externo.
Pathlib	stdlib	Módulo estándar. Detecta automáticamente si el archivo `hero.png` existe en la carpeta del proyecto (admite .png, .jpg, .jpeg, .webp).
io	stdlib	Módulo estándar. Permite generar el archivo Excel en memoria (buffer) antes de entregarlo al usuario para descarga, sin escribir archivos temporales.
math / datetime / date	stdlib	Módulos estándar usados para redondeo, cálculo de fechas y formateo de la fecha actual en la interfaz.
GitHub	—	Plataforma de control de versiones. El código fuente se aloja en un repositorio público de GitHub, lo que permite el despliegue automático en Streamlit Cloud.
Streamlit Cloud	—	Servicio de despliegue gratuito. Lee el repositorio de GitHub, instala las dependencias del archivo `requirements.txt` automáticamente y publica la app en una URL pública accesible sin costo.
CSS personalizado	—	Estilos visuales inyectados mediante `st.markdown()` con `unsafe_allow_html=True`. Implementa el diseño Neon Brutalist: fondo negro, tipografía Space Grotesk, paleta de colores y layout del hero.

7.3 Arquitectura del código

7.3.1 Estructura general del archivo principal

El archivo `finanalisis.py` está organizado en las siguientes secciones en orden de aparición:

1. **Configuración de página**: `st.set_page_config()` con título, ícono y layout ancho.
2. **CSS personalizado**: más de 400 líneas de estilos inyectados al inicio que definen toda la apariencia visual de la app.
3. **Constantes globales**: diccionarios de monedas, listas de categorías de ingresos y gastos, paletas de colores por categoría y layout de gráficas.
4. **Inicialización del estado de sesión**: función `_init()` que define todos los valores iniciales en `st.session_state`.
5. **Funciones helper**: navegación (`ir_a`), formateo de moneda (`fmt`), cálculos de totales (`total_i`, `total_g`, `balance`, `pct_ahorro`).
6. **Funciones financieras**: cálculo de cuota (`cuota_francesa`), conversión de tasas (`tna_a_tea`), TCEA por Newton-Raphson (`calcular_tcea`), tabla de amortización (`tabla_amortizacion`), semáforo de riesgo (`semaforo_tcea`).
7. **Componentes reutilizables**: barra de navegación (`topnav`), selector de moneda (`selector_moneda`).
8. **Páginas**: Inicio, Calculadora de Préstamos, Comparador de Bancos, Gestor de Finanzas Personales. Cada página se activa mediante condicionales sobre `st.session_state.pagina`.

7.3.2 Navegación entre páginas

Streamlit no tiene un sistema de rutas nativo. La navegación se implementó usando `st.session_state` como variable de control.

7.3.3 Privacidad de datos por sesión

Streamlit asigna un `session_state` independiente a cada usuario que abre la app en su navegador. Todos los movimientos del Gestor de Finanzas se almacenan exclusivamente en ese estado de sesión, lo cual garantiza que:

- Ningún usuario puede ver los datos de otro.
- No existe una base de datos compartida.
- Los datos se borran al cerrar la pestaña (privacidad por diseño).
- El usuario puede exportar sus datos como CSV o Excel antes de cerrar.

7.4 Fundamentos de programación aplicados

7.4.1 Funciones

Se definieron **16 funciones** independientes para separar responsabilidades. Cada cálculo financiero tiene su propia función, lo que facilita la verificación y el mantenimiento del código.

Función	Qué hace
`cuota_francesa()`	Calcula la cuota mensual fija por el método francés
`tna_a_tea()`	Convierte la Tasa Nominal Anual a Tasa Efectiva Anual
`calcular_tcea()`	Calcula la TCEA real mediante Newton-Raphson
`tabla_amortizacion()`	Genera el DataFrame con la amortización mes a mes
`semaforo_tcea()`	Devuelve el color y mensaje del semáforo de riesgo
`total_i()` / `total_g()`	Suma ingresos o gastos de una lista de transacciones
`balance()`	Calcula el balance neto (ingresos − gastos)
`pct_ahorro()`	Calcula el porcentaje de ahorro sobre ingresos totales
`ir_a()`	Navega entre páginas actualizando el session state
`topnav()`	Renderiza la barra de navegación superior
`selector_moneda()`	Muestra el selector de moneda y actualiza el estado
`fmt()`	Formatea un número con el símbolo de moneda actual

7.4.2 Condicionales

Se usan condicionales `if / elif / else` en múltiples contextos:

- **Navegación**: determinar qué página mostrar según `st.session_state.pagina`.
- **Semáforo TCEA**: clasificar el riesgo en tres niveles (verde $\leq 15\,\%$, amarillo $\leq 30\,\%$, rojo $> 30\,\%$).
- **Alertas de gasto**: activar la alerta si un gasto supera el 30 % de los ingresos totales.
- **Salud financiera**: mostrar el mensaje y color del indicador mensual según el porcentaje de ahorro.
- **Carga de imagen**: detectar si `hero.png` existe y, en caso contrario, mostrar un placeholder informativo.
- **Edición de movimientos**: mostrar el formulario en modo "editar" o modo "nuevo" según el estado de `editando`.

7.4.3 Bucles

Los bucles `for` se usan en:

- **Algoritmo de Newton-Raphson**: un bucle de hasta 300 iteraciones que converge a la tasa mensual real para el cálculo de la TCEA.
- **Tabla de amortización**: iteración mes a mes para calcular capital, interés y saldo restante.
- **Comparador**: iteración sobre los 3 bancos para calcular y mostrar KPIs, semáforo y gráficas de cada uno.
- **Lista de movimientos**: iteración sobre las transacciones filtradas para renderizar cada tarjeta con sus botones de editar y eliminar.

- **Resumen mensual**: iteración sobre los meses únicos del historial para construir el resumen de cada período.

7.4.4 Listas y diccionarios

- **Listas**: las transacciones del Gestor de Finanzas se almacenan como una lista de diccionarios en `st.session_state.txs`. Cada elemento tiene claves `tipo`, `descripcion`, `monto`, `categoria`, `fecha`, `nota`, `moneda`.
- **Diccionarios**: se usan para mapear categorías a colores (`COLOR_CAT`), categorías a íconos (`ICON_CAT`), y monedas a símbolos (`MONEDAS`).
- **Comprensión de listas**: se aplica para filtrar transacciones por tipo, categoría, mes o texto de búsqueda en tiempo real.

7.4.5 Manejo de excepciones

Dentro del algoritmo de Newton-Raphson se capturan las excepciones `OverflowError` y `ZeroDivisionError` para evitar que la app se rompa ante valores extremos o divisiones por cero durante la iteración numérica.

7.4.6 Módulos estándar de Python

- `base64`: codificación de la imagen hero en base64.
- `pathlib.Path`: detección del archivo de imagen en el sistema de archivos.
- `io.BytesIO`: buffer en memoria para generar el Excel sin escribir archivos temporales en el servidor.
- `datetime.date`: obtención de la fecha actual para el pie de página del gestor y el nombre de los archivos exportados.
- `math`: funciones matemáticas de soporte.
- `json`: disponible para futuras extensiones de persistencia.

7.5 Módulos de la aplicación

7.5.1 Módulo 01 — Calculadora de Préstamos

Permite calcular la cuota exacta, la TEA y la TCEA real de cualquier préstamo ingresando el monto, la tasa nominal anual, el plazo en meses, y los costos adicionales opcionales (seguro de desgravamen y comisiones mensuales).

7.5.2 Módulo 02 — Comparador de Bancos

Permite comparar hasta 3 entidades financieras de forma simultánea, calculando la TCEA real para cada una e identificando automáticamente la opción más conveniente.

7.5.3 Módulo 03 — Gestor de Finanzas Personales

Herramienta completa de seguimiento de ingresos y gastos con análisis visual, alertas automáticas y exportación de datos.

Funcionalidades:

- Registro de movimientos: tipo (ingreso/gasto), descripción, monto, categoría, fecha, nota opcional.
- Edición y eliminación de movimientos ya registrados.
- **Alertas corregidas**: el porcentaje de gasto por categoría se calcula sobre los *ingresos totales* (no sobre el total de gastos). La alerta se activa cuando una categoría supera el **30 % de los ingresos**.
- **Meta de ahorro** con barra de progreso dinámica.
- KPIs en tiempo real: ingresos totales, gastos totales, balance y % ahorro.
- **5 pestañas de funciones**:
  1. **Registrar**: formulario con previsualización del movimiento.
  2. **Movimientos**: historial filtrable por tipo, categoría, texto libre y mes.
  3. **Análisis**: gráfica de ingresos vs. gastos, gastos por categoría y evolución temporal.
  4. **Por Mes**: resumen mensual, indicador de salud financiera y top de gastos con porcentaje sobre ingresos.
  5. **Exportar**: descarga en CSV y Excel (.xlsx con 2 hojas).

7.6 El cálculo de la TCEA — Método matemático

7.6.1 Planteamiento del problema

La TCEA se obtiene encontrando la tasa mensual t_m que iguala el valor presente de todos los pagos reales (cuota base más costos adicionales) con el monto original del préstamo:

$$\text{Monto} = \sum_{t=1}^{n} \frac{\text{Pago mensual real}}{(1+t_m)^t}$$

donde Pago mensual real = Cuota + Seguro + Comisión.

Esta ecuación no tiene solución analítica, por lo que se resuelve numéricamente.

7.6.2 Algoritmo de Newton-Raphson

Se aplica el método iterativo de Newton-Raphson para encontrar t_m:

$$t_m^{(k+1)} = t_m^{(k)} - \frac{f(t_m^{(k)})}{f'(t_m^{(k)})}$$

donde:

$$f(t_m) = -\text{Monto} + \sum_{t=1}^{n} \frac{P}{(1+t_m)^t} \qquad f'(t_m) = \sum_{t=1}^{n} \frac{-t \cdot P}{(1+t_m)^{t+1}}$$

El algoritmo itera hasta 300 veces o hasta que la diferencia entre iteraciones sea menor a 10^{-12}. Se capturan `OverflowError` y `ZeroDivisionError` para garantizar estabilidad numérica.

7.6.3 Anualización de la tasa

Una vez obtenida la tasa mensual convergida t_m, se anualiza:

$$\boxed{\text{TCEA} = (1 + t_m)^{12} - 1}$$

7.6.4 Validación de los resultados

Pruebas unitarias realizadas con Python (ast + cálculo directo)

Caso de prueba	Resultado verificado
Cuota: S/10,000 · TNA 18 % · 24m	S/ 499.24 ✓
TEA de TNA 18 %	19.56 % ✓
TCEA sin costos adicionales	TCEA = TEA ✓
TCEA con S/25 seguro + S/15 comisión	29.46 % > TEA ✓
Saldo final de amortización	0.0000 ✓

7.7 Diseño visual

7.7.1 Estética Neon Brutalist

El diseño sigue la corriente **Neon Brutalist**, caracterizada por fondos muy oscuros, tipografía pesada y colores neón de alto contraste como acentos:

Elemento	Valor	Uso
Fondo principal	`#060606`	Toda la app
Fondo de tarjetas	`#0A0A0A`	Cards y secciones
Acento rosa	`#FF3366`	Botones, módulo 01, alertas
Acento amarillo	`#EEFF00`	Hero, módulo 02, highlights
Acento cyan	`#00FFCC`	Módulo 03, ingresos
Tipografía	Space Grotesk 700–900	Todos los textos
Texto principal	`#F1F5F9`	Títulos y valores
Texto secundario	`#BBBBBB`	Descripciones

7.7.2 Hero section con imagen

La pantalla de inicio utiliza un **layout de dos columnas**: el texto del hero a la izquierda y la imagen `hero.png` a la derecha. La imagen se codifica en base64 con el módulo estándar de Python y se incrusta directamente en el HTML sin depender de un servidor de archivos externo.

7.7.3 Implementación del CSS

El CSS personalizado se inyecta mediante `st.markdown()` con `unsafe_allow_html=True` al inicio de la app. Incluye:

- Importación de la fuente Space Grotesk desde Google Fonts.
- Estilos para todos los componentes nativos de Streamlit (inputs, selectboxes, botones, tabs, métricas, dataframes).
- Clases personalizadas para tarjetas, badges numéricos, panel TCEA, semáforos, barra de navegación y hero section.
- Animación CSS (keyframes) para el punto parpadeante del tagline.

7.8 Despliegue e instrucciones de uso

7.8.1 Archivos del proyecto

Archivo	Descripción
`finanalisis.py`	Archivo principal. Contiene toda la lógica, cálculos, interfaz y CSS. Aproximadamente 1,750 líneas.
`requirements.txt`	Lista de dependencias con versiones mínimas. Streamlit Cloud lo lee para instalar todo automáticamente.
`hero.png`	Imagen de la sección hero. La app la detecta automáticamente si está en la misma carpeta.

Contenido del archivo `requirements.txt`:

```
streamlit>=1.32.0
pandas>=2.0.0
numpy>=1.24.0
plotly>=5.18.0
openpyxl>=3.1.0
```

7.9 Video de presentación

El siguiente video muestra el funcionamiento completo de la aplicación: la pantalla de inicio, los tres módulos en acción, el cálculo de la TCEA con semáforo de riesgo, la comparación de bancos y el gestor de finanzas personales con sus herramientas de análisis y exportación.

Enlace al video en YouTube

Título: FinAnalysis: La herramienta para calcular el costo real de préstamos y gestionar tus finanzas

URL: https://youtu.be/QT2V7mOsFzM **Duración:** 6 minutos 15 segundos

7.10 Conclusión

Finanálisis demuestra que es posible construir una herramienta financiera completa, matemáticamente precisa y visualmente atractiva usando exclusivamente tecnologías de código abierto y gratuitas.

El proyecto aplica de forma integrada los fundamentos del lenguaje de programación Python: funciones, condicionales, bucles, listas, diccionarios, manejo de excepciones y módulos estándar. Estos elementos no se presentan de forma aislada sino como piezas de una aplicación real que resuelve un problema concreto.

Finanálisis

Decide con datos, no con suposiciones

`finanalisis-app.streamlit.app` · Puno, Perú · 2026

Lista de Ejercicios

1. Complejidad temporal — Bucles anidados ($O(n^2)$, $n = 100$)
2. Búsqueda binaria — $O(\log n)$, máximo de comparaciones
3. Comparativa A/B/C — $O(n)$ vs $O(n^2)$ vs $O(1)$
4. Fibonacci recursivo — $O(2^n)$, ineficiencia
5. Eliminar duplicados — temporal $O(n^2)$, espacial $O(n)$
6. Bucle logarítmico — potencias de 2
7. Bubble Sort vs Quick Sort — peor caso
8. Número primo — $O(\sqrt{n})$
9. Multiplicación de matrices — $O(n^3)$ vs Strassen
10. Tabla de complejidades por estructura de datos
11. Sistema de registro de temperaturas (escritura + append)
12. Base de datos de productos con menú (archivo binario)
13. Lectura de 100 archivos y promedio de edades
14. Lectura por bloques de 10 archivos
15. Registro de ventas diarias en archivos separados
16. Total de ventas desde múltiples archivos
17. Temperaturas por hora en archivos separados
18. Temperatura máxima del día
19. Logs de errores por tipo (append)
20. Contar errores en archivo de log
21. Backup de archivo carácter por carácter
22. Filtrar temperaturas mayores a 20°C
23. Consolidar reportes de 3 sucursales
24. Procesar CSV de calificaciones
25. Ficha de mascota con `struct`
26. Registro de temperatura — 3 registros con promedio
27. Salón de clases — promedio y mejor alumno
28. Inventario — valor total y stock bajo
29. Boleta de pago con descuento AFP
30. Sensores guardados en archivo de texto
31. Biblioteca — completar código (struct + archivos)

32. Temperaturas mensuales — completar funciones

33. Corregir inventario — 4 errores

34. Refactorizar con función `calcular_promedio()`

— Fin de la Segunda Unidad —

Lenguaje de Programación I UNAP 2026